

A Data-Driven Capacity-Reservation System for Cloud Cost Optimization under Intermittent Demand Surges

First Author, Second Author, and Third Author

Abstract—Conservative fractile heuristics—sizing capacity at a high percentile of observed utilization plus headroom—are standard practice in cloud right-sizing, while operations-management theory prescribes newsvendor-fractile sizing of reserved capacity under two-tier pricing. What this conservatism costs when carried over to reservation sizing has been, to our knowledge, unquantified. We bridge practice and theory. Smart Cloud Optimizer operationalizes the capacity-reservation policy as a pipeline of walk-forward-validated forecasting, mixed-integer right-sizing at an empirical $q_{0.95} \times 1.3$ fractile, and reservation actions gated on always-on usage; on a 365-day *synthetic* AWS account it emits 19 recommendations worth 27.0% of the \$2189.48/month bill. An out-of-sample policy simulation then prices the heuristic: at the catalog reserved/on-demand ratio $\gamma=0.63$, the deployed fractile—read counterfactually as a reservation-sizing rule—would realize 101.8% (synthetic) and 153.5% (real trace) of pure on-demand cost, forfeiting the entire discount. The newsvendor fractile instead realizes 73.6%/86.2%, and a two-tier base/supplementary policy 72.8%/77.7%; a causal variant retains nearly all of that gain on real demand. On *real* telemetry (2,297 VMs from the Bitbrains and Materna public traces), mixed-integer right-sizing over a verified AWS catalog saves 33.1–74.2% against a like-for-like lift-and-shift baseline, rising to 58.0–83.8% with one-year reservations; on a 237,272-VM Azure fleet with max-based percentiles and unmeasured memory, right-sizing alone turns cost-negative while reservations still save 34.8%. Point forecasts never significantly beat seasonal-naïve baselines, and real-aggregate forecast error never falls below 48.2% MAPE—supporting sizing from demand *quantiles*. Every number is script-regenerated from raw data to compiled PDF. The deployed always-on reservation gate avoids the penalty we measure.

Index Terms—cloud cost optimization, capacity reservation, newsvendor, reserved instances, time-series forecasting, mixed-integer programming, FinOps.

I. INTRODUCTION

PUBLIC-CLOUD adoption continues to accelerate, but cost governance has not kept pace: industry surveys consistently report that roughly a quarter to a third of cloud spend is wasted on idle or over-provisioned resources [1], [2], hyperscaler telemetry shows chronic over-provisioning at the VM level [3], [4], and a recent production trace of 48,000 enterprise VMs found more than 80% using under 70% of their provisioned resources [5]. The root cause is a mismatch between how demand behaves and how capacity is priced. Demand for a typical workload is a *base* load—low, stationary,

and predictable—punctuated by intermittent *surges* driven by promotions, launches, batch jobs, or seasonal events [6]–[8]. Pricing, meanwhile, is two-sided: a *reserved* (committed) tariff that is cheap per unit but is paid whether used or not, and an *on-demand* tariff that is elastic with zero commitment but costs substantially more per unit. Choosing how much capacity to reserve versus leave to on-demand is therefore a classic trade-off between commitment risk and elasticity premium.

Chen, Lei, and Moinszadeh [9] formalize this trade-off as a capacity-reservation problem. A firm holds *base contracts* sized to the stationary base demand and *supplementary contracts* for surges; demand beyond reserved capacity spills to on-demand. They show that the cost-optimal capacities follow a *newsvendor critical fractile*, and that the optimal management of supplementary contracts (purchase, renewal, cancellation, expiration) is a two-threshold (s, S) policy. This is a closed-form *characterization*, but it presumes the demand distributions are known and deterministic in structure, and it produces a policy rather than an operational pipeline that an engineering team can run against a real account.

A parallel systems literature attacks the same waste empirically—forecasting utilization and reacting with autoscaling or instance acquisition [10]–[15]—but rarely connects its heuristics back to the cost-optimal reservation policy, and seldom reports end-to-end dollar savings with a reproducible methodology. The two bodies of work are complementary: one supplies the economic optimum, the other supplies the data and tooling.

This paper bridges the two. We present *Smart Cloud Optimizer*, a system that operationalizes the base/supplementary/on-demand reservation framework of [9] on account telemetry, and—new relative to prior operationalizations—we measure, in both directions, what the bridge is worth: what the theory gains from real data, and what deployed practice would gain from the theory. Our contributions are:

- 1) **An end-to-end system realizing a reservation policy** (Section IV): a mixed-integer linear program (MILP) sizes capacity at a fixed empirical fractile (the 95th percentile of observed utilization with $1.3 \times$ headroom—the same percentile idiom used by production limit-setters such as Google Autopilot [15] and Azure’s recommenders [3], [16]), eight rules choose reserved-vs-on-demand contracts and remove service-level waste, and five implemented time-series forecasters (four compared by walk-forward cross-validation; SARIMAX excluded for runtime) provide demand monitoring, prediction

The authors are with the Department of Scientific Computing, Faculty of Computer and Information Sciences, *Affiliation Placeholder*, City, Country (e-mail: {author1, author2, author3}@example.edu).

Manuscript received July 2026. (*Author block is a placeholder; fill before submission.*)

intervals, and surge context—sizing itself uses empirical quantiles of the telemetry.

- 2) **External validation on real telemetry** (Section VI): on 2,297 VMs from two providers—the Bitbrains GWA-T-12 and Materna GWA-T-13 public traces [7], [17]—priced against a verified, snapshot-dated AWS catalog, the system’s MILP cuts a like-for-like lift-and-shift baseline by 33.1%–74.2% (58.0%–83.8% with one-year reservations): nearly identical on the two independent Bitbrains datasets (33.6%/33.1%) and larger on the far more over-provisioned Materna fleet. A 237,272-VM Azure fleet exposes a boundary condition: max-based percentiles and unmeasurable memory turn right-sizing alone cost-negative (−3.8%) while reservations still yield 34.8%. The same study shows honestly that the *aggregate* demand of a real cluster is barely point-forecastable at its native hourly scale (best mean absolute scaled error, MASE, ≈ 0.91 – 0.93 at the 24-hour horizon), which argues for sizing from demand *quantiles* rather than point forecasts.
- 3) **Quantifying the price of conservatism, out of sample** (Section VII): a policy simulation on held-out demand shows that at the measured reserved discount ($\gamma=0.63$), the deployed $q=0.95 \times 1.3$ fractile—if read as a *reservation-sizing rule*—realizes 101.8% (synthetic) and 153.5% (real trace) of pure on-demand cost, i.e. it forfeits the entire discount, whereas the newsvendor fractile $1-\gamma$ [18], [19] realizes 73.6% and 86.2%. A sample-average-approximation stochastic program [10], [20] recovers the closed-form quantile to within 0.57% (mutual validation), and a surge-aware dual-capacity (K_0, K_1) policy—the first step toward the (s, S) dynamics of [9]—is worth little on tame synthetic demand (+0.8 points) but +8.5 points on real bursty demand, of which a *causal* one-period activation rule retains +8.1: empirical evidence that the two-tier contract structure earns its complexity exactly when surge states persist long enough to be acted on—the regime it was designed for.
- 4) **A statistically honest, fully regenerable evaluation** (Sections V–VII): forecast accuracy is reported as mean $\pm$ sd over walk-forward folds with MASE [21], interval coverage, and Diebold–Mariano tests [22], [23]; where a difference is not significant (ETS vs. seasonal-naive at the 30-day horizon, $p=0.198$) we say so and temper the claim. Every figure, table cell, and in-text number is emitted by a script from the database, the public traces, or the price catalog; no number in this paper is hand-typed.

Sections II–IV position the work, restate the reservation model, and describe the system; Sections V–VII report the three studies; Sections VIII–IX discuss threats and conclude.

II. RELATED WORK

We organize related work into four streams: the operations-management (OM) reservation stream our policy comes from, the cloud cost-optimization and commitment stream, the

production right-sizing stream our heuristic comes from, and the forecasting stream our estimators come from.

(1) Capacity reservation under random demand and surges. The reservation/option-contract view of capacity has a long history in operations: backup and reservation contracts trade commitment for flexibility [24], [25], and planning for random demand surges combines reactive capacity with safety stock [26]—an idea mirrored in service operations, where committed staff are blended with flexible on-demand labor [27]. State-modulated (base-vs-surge) demand is modeled classically via Markov-modulated processes in inventory and queueing [28], [29], and the optimal single-period commitment is the newsvendor critical fractile [18]; Ban and Rudin [19] give the modern data-driven treatment in which the fractile is estimated directly from samples, which is exactly the estimator our simulation uses. Our modeled reference [9] specializes the reservation problem to cloud reserved/on-demand pricing with cancellation and a secondary marketplace; follow-on OM work studies data-driven pricing of upfront reservation-discount contracts [30] and provider-side capacity under periodic demand [31]. We borrow the *policy* from this stream and ask what it is worth on real telemetry.

(2) Cloud cost optimization, commitment portfolios, and instance acquisition. On the cloud-user side, stochastic programming chooses reservation levels under demand and price uncertainty [10], [20]; online and brokerage algorithms acquire reserved/on-demand instances over time [11], [32]; and the reserved-vs-on-demand decision has a clean theoretical core in (learning-augmented, multi-option) ski-rental problems [33], [34]. Scheduling and instance switching cut rental cost [12]; marketplace reselling hedges over-reservation [35]; platform-side work studies overcommitment [36], slack capacity [37], bundled procurement [38], and preemptible discounts [39], [40]. Closest to us as *systems* are end-to-end FinOps pipelines: ABACUS enforces budgets and pre-deployment cost checks in a FinOps pipeline [41], Alves et al. combine prediction with pricing-model selection [42], and Musa et al. operationalize a reservation model with a single regression forecaster [43]; recent reviews map the wider FinOps research gap [44]. Commercial recommenders document the same historical-replay principle: AWS simulates eligible reservation combinations against a 7–60-day usage look-back and recommends the savings-maximizing purchase, explicitly assuming history reflects the future [45], [46], and Azure evaluates hourly usage over the same windows to recommend the savings-maximizing quantity [47]—a computation that, in the single-resource abstraction of Section VII, reduces to the sample-average stochastic program we use as a baseline. Our delta is deriving the reserved fractile from the cost-optimal policy of stream (1), quantifying the gap to deployed heuristics out of sample, and validating the pipeline on public traces.

(3) Right-sizing and over-provisioning in production. Percentile-based limit setting is the de facto industrial standard: Google Autopilot sets container limits from usage percentiles—autopiloted jobs run at 23% slack versus 46% for manually-managed ones [15]; Azure’s Resource Central predicts utilization percentiles to drive allocation decisions [3], and Doppler recommends SKUs from telemetry [16]. Academic work selects

TABLE I
MODEL NOTATION (AFTER [9]).

Symbol	Meaning
w_0	on-demand cost rate per unit capacity per period
w_T	reserved cost rate, $w_T < w_0$ (commitment of length T)
γ	reserved/on-demand ratio $w_T/w_0 \in (0, 1)$
d_0, d_1	base / surge demand per period, d_1 dominates d_0
F_0, F_1	CDFs of base and surge demand
t_d, t_s	surge duration / inter-surge duration
K_0	base capacity (reserved indefinitely)
K_1	total capacity during a surge; $K_1 - K_0$ supplementary
$(x)^+$	$\max(x, 0)$, the on-demand spillover operator
α	fraction of reserved value recovered on cancellation

instance types from benchmarked performance [48], formulates VM allocation as integer programs [49], and quantifies the requested-vs-used gap on public traces [4], [5], [50], [51]. Our MILP (Section IV) belongs to this family—its 95th percentile with headroom is deliberately the deployed idiom—and our contribution over this stream is to read that idiom through the reservation optimum and measure the cost of the difference (Section VII).

(4) Forecasting-driven provisioning. Time-series forecasting underpins proactive provisioning [13], [14], [52], [53], and forecast-then-optimize pipelines run in production at hyperscalers [54], [55]; autoscaling-policy choice alone materially changes realized cost in microservice deployments [56]. We use established models—exponential smoothing/Holt–Winters [57], [58], automatic ARIMA [59], and Prophet [60] via statsmodels [61]—together with streaming anomaly detection in the spirit of the Numenta Anomaly Benchmark [62]. Deep forecasters are well represented in cloud-workload prediction [63], [64], and time-series foundation models are emerging [65]–[67], including pretraining on cloud-operations data [68]. We deliberately evaluate classical models: on long-horizon benchmarks simple linear models remain competitive with Transformer architectures [69], comparative studies on real cloud traces reach similar conclusions [70], and our single-account data regime (hundreds of daily points) offers little to pretrain-scale methods; testing foundation models is future work. To our knowledge, none of the forecasting-driven provisioning works we surveyed ties the forecast to a *newsvendor* reservation fractile and quantifies the fractile’s cost consequences out of sample, which is our integrative aim.

III. RESERVATION MODEL AND POLICY

We restate the model of [9] in the notation of Table I, then derive the sizing principle our system approximates. Section VII measures what ignoring it costs. (We use *fractile* q , the q -quantile, and the $100q$ -th percentile interchangeably.)

Cost structure. A firm serves a state-modulated demand: a stationary base state (demand d_0 , fraction $\frac{t_s}{t_s+t_d}$ of time) and a surge state (demand d_1 , fraction $\frac{t_d}{t_s+t_d}$). Holding base capacity K_0 and surge capacity K_1 , the long-run average cost rate is

reserved cost plus on-demand spillover priced at w_0 :

$$\begin{aligned} \text{AC}(K_0, K_1) = & \underbrace{w_T K_0}_{\text{base}} + \underbrace{w_T (K_1 - K_0) \frac{t_d}{t_s+t_d}}_{\text{supplementary}} \\ & + w_0 \frac{t_s}{t_s+t_d} \mathbb{E}(d_0 - K_0)^+ \\ & + w_0 \frac{t_d}{t_s+t_d} \mathbb{E}(d_1 - K_1)^+. \end{aligned} \quad (1)$$

The first two terms are committed reserved spend; the last two are the expected *on-demand spillover*, the demand that exceeds reserved capacity.

Newsvendor optimality. Minimizing (1) yields a critical-fractile condition. In the single-capacity case ($K_0 = K_1 = K$) the optimum K^* solves [9, Prop. 1]

$$F_0(K^*) + \frac{t_d}{t_s} F_1(K^*) = \frac{w_0 - w_T}{w_0} \left(1 + \frac{t_d}{t_s}\right), \quad (2)$$

which is the first-order condition of (1) rescaled by $(t_s+t_d)/t_s$. In the dual-capacity case the optimal base and surge capacities are newsvendor quantiles of F_0 and F_1 ,

$$K_0^* = F_0^{-1}(q_0), \quad K_1^* = F_1^{-1}(q_1), \quad (3)$$

with fractiles q_0, q_1 that depend on w_T/w_0 , the cancellation recovery α , and the duration ratio [9, Thm. 1]. In [9], managing the supplementary contracts over time is a two-threshold (s, S) policy on the elapsed cycle time: cancel below a lower threshold, hold to expiration above an upper threshold, and otherwise renew.

Remark 1 (Service level reading). *Ignoring the surge term, (2) reduces to the textbook newsvendor $F(K^*) = 1 - w_T/w_0 = 1 - \gamma$: the optimal reserved capacity covers the $(1 - \gamma)$ quantile of demand. Cheaper reserved capacity (small γ) means reserve more; reserved capacity that is barely discounted (large γ) means reserve little and let demand spill to on-demand.*

From policy to an empirical fractile. Our system does not know F_0, F_1 analytically; it estimates them from telemetry. For a resource i with observed per-period utilization u_i , the deployed right-sizing rule sizes the required capacity to

$$r_i = q_{0.95}(u_i) \cdot V_i \cdot h, \quad h = 1.3, \quad (4)$$

where $q_{0.95}(u_i)$ is the 95th percentile of utilization, V_i the current resource size, and h a headroom factor. Equation (4) has the form of a *critical-fractile* capacity choice with implied service level $q \approx 0.95$ —the same percentile idiom used by production limit-setters [3], [15]. Reading it through Remark 1, a 0.95 service level is cost-optimal for *reservation* sizing only when $\gamma \approx 0.05$ (near-free reserved capacity); for realistic reserved discounts ($\gamma \in [0.6, 0.9]$) the cost-optimal fractile is 0.10–0.40. The deployed heuristic is therefore deliberately *conservative*: it favors avoiding performance-impacting underprovisioning over extracting the last dollar. We make this gap explicit rather than claim the heuristic is cost-optimal—and in Section VII we measure, out of sample on both a synthetic account and a real trace, exactly what the conservatism costs when the fractile is used to size reserved capacity.

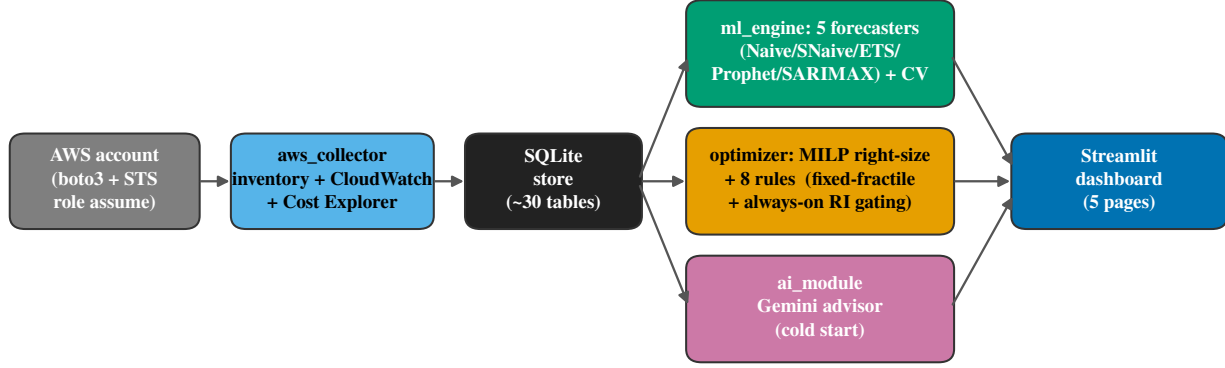


Fig. 1. System architecture. Telemetry is collected per AWS service, stored in SQLite, and consumed by three engines—forecasting, the reservation optimizer (MILP + rules), and an LLM advisor—and surfaced in a dashboard. The optimizer realizes the static core of the reservation policy of Section III (fixed-fractile sizing plus always-on reservation gating); Section VII quantifies its distance from the newsvendor/(s, S) optimum.

IV. SYSTEM: FORECASTING + MILP + RULES

Fig. 1 shows the pipeline. A collection layer gathers inventory, CloudWatch metrics, and Cost Explorer data for ten AWS resource types (EC2/EBS, RDS, S3, Lambda, DynamoDB, ElastiCache, ECS, ELB, NAT) into a per-account SQLite store; three engines then consume it.

A. Forecasting the base demand distribution

The forecasting engine implements five models behind one interface emitting $[date, forecast, lower, upper]$: a naive last-value baseline, a seasonal-naive (weekly) baseline, additive Holt–Winters exponential smoothing (ETS) [57], [58], automatic seasonal ARIMA (SARIMAX) [59], and Prophet [60]. Models are compared by *walk-forward (rolling-origin) cross-validation* [71], [72]: an expanding training window of at least 120 days, a step of 30 days, and a forecast horizon h ; accuracy is averaged over folds. The selected model provides trend forecasts and prediction intervals for monitoring; the empirical base distribution $\hat{F}_0$ used for sizing is taken directly from the observed telemetry, and points exceeding $\mu + 2\sigma$ are flagged as surges (the realization of the state-1 / d_1 regime), in the spirit of streaming anomaly detection [62].

B. Sizing capacity with a MILP

For compute (EC2, RDS) the optimizer solves a mixed-integer program with CBC via PuLP [73]. Let $\mathcal{I}$ be the in-scope instances, $\mathcal{C}$ the catalog of candidate types with price p_c , vCPUs v_c , and memory m_c , and $x_{i,c} \in \{0, 1\}$ assign type c to instance i :

$$\begin{aligned}
 \min_x \quad & \sum_{i \in \mathcal{I}} \sum_{c \in \mathcal{C}} p_c x_{i,c} \\
 \text{s.t.} \quad & \sum_c x_{i,c} = 1 & \forall i \in \mathcal{I}, \\
 & \sum_c v_c x_{i,c} \geq r_i^{\text{cpu}} & \forall i, \\
 & \sum_c m_c x_{i,c} \geq r_i^{\text{mem}} & \forall i, \\
 & \sum_i \sum_c p_c x_{i,c} \leq B, \quad x_{i,c} \in \{0, 1\},
 \end{aligned} \tag{5}$$

where the capacity requirements $r_i^{\text{cpu}}, r_i^{\text{mem}}$ are the empirical fractiles of (4) and B is an optional budget cap (left unbounded in our runs, so it does not shape the reported savings). The memory constraint is imposed only for instances exposing a comparable memory-utilization metric (EC2; omitted for RDS). The objective picks the cheapest catalog type whose vCPU and memory cover the fractile-sized load: this is the computational analogue of choosing K_0 in (3), with the residual tail $(d - r_i)^+$ left to on-demand spillover.

C. Choosing reserved vs. on-demand, and removing waste

Eight rule-based actions complete the policy (Algorithm 1). Two *pricing* rules realize the base-contract decision: an on-demand resource with sufficient usage history is switched to a one-year reservation when the reserved rate beats on-demand—i.e., committing the always-on base capacity while leaving variable load on-demand. The pricing catalog’s one-year reserved rate is 37% below on-demand ($\gamma \approx 0.63$, and current AWS one-year standard no-upfront pricing gives the same ratio, $\gamma \approx 0.63$; Section VI), so the reported reservation savings scale roughly with $1 - \gamma$. The remaining six target service-level waste: EBS gp2 $\rightarrow$ gp3 migration, deletion of unattached/idle volumes and idle load balancers, NAT-gateway $\rightarrow$ VPC-endpoint replacement, S3 intelligent-tiering, DynamoDB capacity-mode switching, and Lambda memory right-sizing. Each action emits a per-resource saving $\Delta_i = c_i^{\text{cur}} - c_i^{\text{new}}$; the engine de-duplicates by (resource, action) and persists the recommendations.

A cold-start advisor (a large-language-model prompt over a nine-question requirements form) covers accounts with no usage history; it is orthogonal to the reservation policy and we do not evaluate its dollar impact here.

V. STUDY I: SYNTHETIC ACCOUNT

A. Setup and reproducibility

Following the structure of [9, Sec. 7], we first fix the workload, then evaluate forecasting and optimization. The workload is a single representative account observed for 365

Algorithm 1 Reservation-policy realization for one account**Require:** account telemetry $\mathcal{D}$, catalog $\mathcal{C}$, budget B

```

1: SELECTMODEL( $\mathcal{D}$ )  $\triangleright$  walk-forward CV; monitoring
2: flag surges where cost  $> \mu + 2\sigma$   $\triangleright$  state-1 demand
3:  $R \leftarrow \emptyset$ 
4: for each compute service  $s \in \{\text{EC2, RDS}\}$  do
5:    $r_i \leftarrow q_{0.95}(u_i) V_i h$  for  $i \in \mathcal{I}_s$   $\triangleright$  Eq. (4)
6:    $x^* \leftarrow \text{solve MILP (5) with } (r_i, B)$ 
7:   add right-size recs where  $x_i^* \neq$  current and cheaper
8: end for
9: for each rule  $\rho$  in the eight actions do
10:   $R \leftarrow R \cup \rho(\mathcal{D})$   $\triangleright$  reserved switch / waste removal
11: end for
12: de-duplicate  $R$  by (resource, action); keep max saving
13: return  $R, \sum_{r \in R} \Delta_r$ 

```

days (2024-07-01 to 2025-06-30), spanning 6 billed service categories and 42 resources (including 8 EC2 instances). Daily cost averages \$74.64 (standard deviation \$22.87, range \$36.10–\$232.52), totaling \$27,243.31 over the year. Two complementary bill views are used: a point-in-time resource-inventory bill (\$2189.48/month, the savings denominator) and this historical daily cost series (\$27,243.31/year). They differ by a few percent because the former is a current snapshot of provisioned resources while the latter integrates a year of daily charges, including transient resources; all savings ratios use the inventory baseline. Defining a *demand surge* as a day exceeding $\mu + 2\sigma = \$120.38$ isolates 5 pronounced spikes (Fig. 2); the largest reaches $z = 6.9\sigma$ above the mean. We read these spikes as the intermittent state-1 (surge) events of the model; they recur irregularly across the year—the regime the reservation policy is designed to handle. EC2 dominates the bill (Fig. 3), so compute right-sizing and reservation carry the largest savings potential. The workload is *synthetic*, modeled on public cloud traces [7], [62]; Section VI repeats the transferable parts of the study on real telemetry.

Reproducibility. Every number, table cell, and figure in Sections V–VII is emitted as a $\text{\LaTeX}$ macro by four scripts in five invocations (`make_figures.py`, `external_validation.py`, `azure_validation.py`, and `policy_sim.py` on two inputs) from the account database, the public traces (fetched by a pinned script from the archive maintainers’ mirror), and a snapshot-dated price catalog. Stochastic components are seeded; we verify by running the chain twice in the pinned environment and diffing every generated artifact.

B. Forecasting performance

Table II and Fig. 4 report walk-forward mean absolute percentage error (MAPE, mean $\pm$ sd across folds) by model and horizon; at the deployment-relevant 30-day horizon we additionally report the mean absolute scaled error (MASE) [21] and 95%-interval coverage. Exponential smoothing (ETS) has the best mean at 30 days ($11.2\% \pm 0.57$) and the best MASE (0.81 vs. 0.89 for seasonal-naïve; both beat the in-sample seasonal-naïve benchmark of 1.0), and its accuracy is far

TABLE II
STUDY I: WALK-FORWARD MAPE (% , MEAN $\pm$ SD ACROSS FOLDS) BY MODEL AND HORIZON (DAYS). BEST MEAN PER COLUMN IN BOLD. EXPANDING WINDOW, INITIAL 120 DAYS, STEP 30 DAYS.

Model	$h=7$	$h=14$	$h=30$	$h=60$	$h=90$
ETS (selected)	11.28 $\pm$ 2.54	11.24 $\pm$ 2.45	11.20 $\pm$ 0.57	11.39 $\pm$ 0.74	12.15 $\pm$ 2.39
SeasonalNaïve	10.18 $\pm$ 2.84	10.84 $\pm$ 2.75	12.28 $\pm$ 2.82	12.66 $\pm$ 2.21	12.91 $\pm$ 2.02
Prophet	14.41 $\pm$ 6.22	20.47 $\pm$ 7.40	20.93 $\pm$ 7.53	19.70 $\pm$ 7.89	18.08 $\pm$ 8.42
Naïve	25.16 $\pm$ 4.47	27.90 $\pm$ 3.52	27.26 $\pm$ 4.36	27.87 $\pm$ 4.09	27.38 $\pm$ 3.70

more stable across folds (sd 0.57 vs. 2.8). However, the point-accuracy difference between ETS and the seasonal-naïve baseline is *not statistically significant* at this sample size: a Diebold–Mariano test with the Harvey–Leybourne–Newbold small-sample correction [22], [23] on the pooled 1–30-step-ahead errors (8 contiguous, non-overlapping test windows; autocovariance truncation at lag $h-1$) gives $DM = -1.29$, $p = 0.198$, and a Wilcoxon signed-rank test on per-fold MAPEs gives $p = 0.250$. We therefore claim model *selection*, not model *superiority*: ETS is selected for its stability, MASE, and conservative, over-covering intervals (99% empirical coverage vs. nominal 95%—over-wide but safe for capacity work, unlike Prophet’s severe under-coverage at 70%), consistent with guidance on comparing few models over short evaluation windows [74], [75]. Prophet trails on accuracy ($20.9\% \pm 7.5$ at 30 days)—its yearly-seasonality components overfit a single year of data—and the naïve baseline (27.3%) confirms the series is non-trivial.

Fig. 5 shows the selected model’s 30-day holdout forecast with a 95% interval: ETS tracks the base load at 12.0% MAPE with 97% empirical interval coverage, but the lone in-window surge pierces the upper band. This is consistent with the policy’s premise: on this workload the base process does not anticipate the surge, so the surge is the spillover the reservation policy routes to on-demand rather than provisioning reserved capacity against. Because the workload is synthetic with injected spikes, we read this as confirming a modeling assumption, not as evidence that real-world surges are inherently unforecastable.

C. Optimization savings

The optimizer produces 19 recommendations (plus 10 advisory items). Summed independently they total \$590.40/month; because 3 EBS volumes are flagged for both deletion and a `gp3` upgrade, the jointly-realizable total after de-duplication is \$588.40/month (the \$2.00 difference is the only double-counted amount). Against the \$2189.48/month inventory bill the additive figure is a 27.0% reduction (upper bound), taking monthly spend to \$1599.08; on the \$1074.83 of resources flagged, the upper-bound reduction is 54.9%. Fig. 6 breaks the savings down by action. Two compute levers dominate and, crucially, target *disjoint* resources—MILP right-sizing of over-provisioned RDS databases (\$362.81, 2 recs) and the reserved/pricing switch on always-on EC2 instances (\$208.42, 6 recs)—so their combined \$571.23 (96.8% of the total) is fully jointly realizable. The six waste-removal rules contribute the long tail. On this EC2/RDS-heavy account the dominance of the two compute levers reflects the workload composition—a

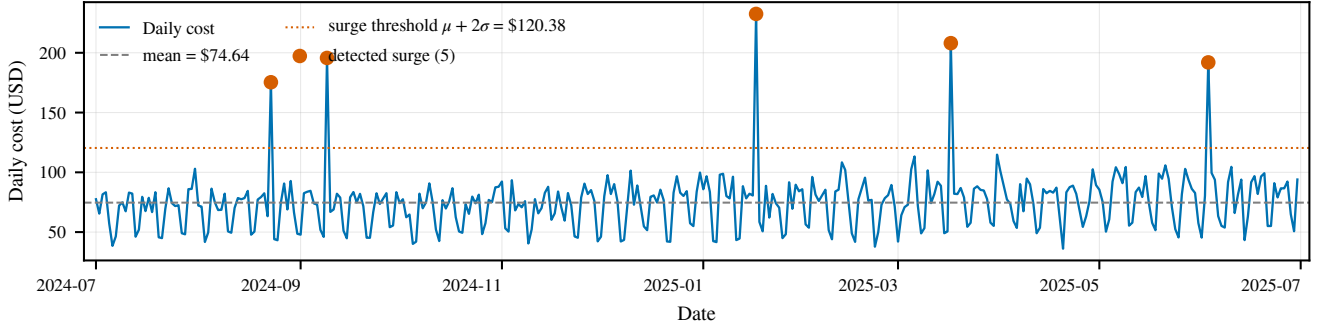


Fig. 2. Daily cost over 365 days (synthetic account). The dotted line is the surge threshold $\mu + 2\sigma = \$120.38$; 5 days (markers) exceed it. Base demand is stationary around \$74.64/day; surges are intermittent and large.

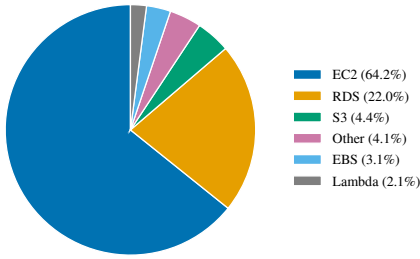


Fig. 3. Study I: share of the monthly bill by service category (Cost Explorer buckets; non-itemized services fold into “Other”). EC2 dominates, so compute right-sizing and reservation carry the largest savings.

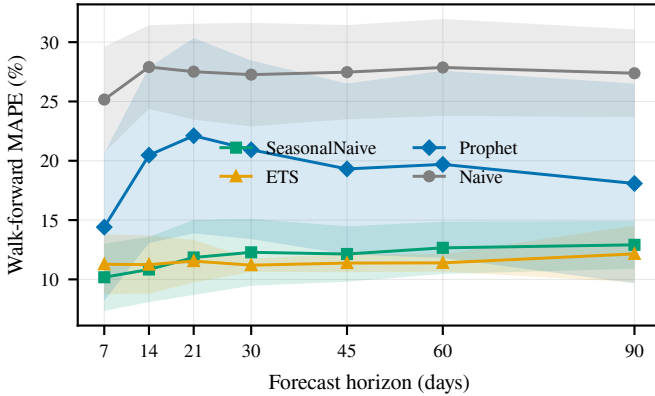


Fig. 4. Study I: walk-forward MAPE vs. horizon (shaded bands: ± 1 sd across folds). ETS has the best mean at $h \geq 30$ days and the smallest fold-to-fold sd; seasonal-naive wins at short horizons, and their difference is not statistically significant (Section V-B). Both beat the naive baseline. SARIMAX is excluded for runtime (minutes per fold).

few large over-provisioned and always-on instances—rather than a theorem; it is consistent with the policy’s emphasis on sizing base capacity correctly. By confidence, 8 high-confidence recommendations carry \$370.52, 9 medium-confidence ones \$214.21, and 2 low-confidence ones \$5.67.

VI. STUDY II: EXTERNAL VALIDATION ON REAL TRACES

Study I establishes the pipeline on a workload whose ground truth we control; this section validates the transferable parts on

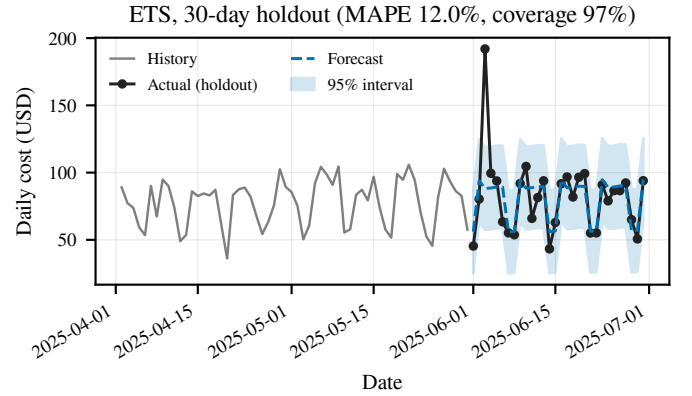


Fig. 5. Study I: ETS 30-day holdout forecast with 95% interval. The base load is tracked at 12.0% MAPE; the single surge exceeds the band (on-demand spillover).

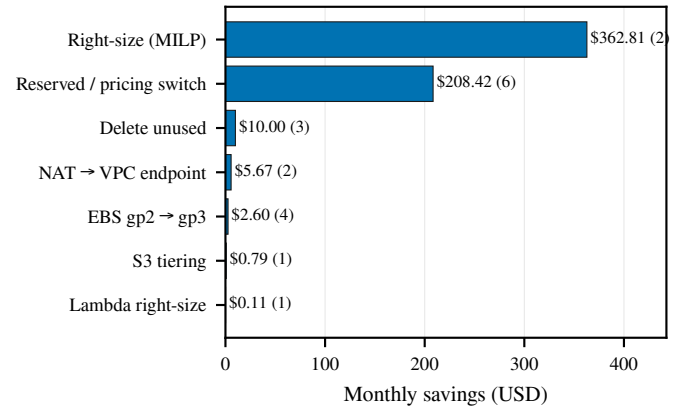


Fig. 6. Study I: recommended monthly savings by optimization action (USD; recommendation count in parentheses). Right-sizing and the reserved/pricing switch dominate.

real telemetry from two managed-hosting providers, and reports honestly where reality is harder than the synthetic account.

A. Traces, cost model, and protocol

We use three public per-VM datasets: **fastStorage** (1250 VMs, one month, August 2013) and **Rnd** (500 VMs, three months) from the Bitbrains GWA-T-12 trace of business-critical

enterprise workloads [7], [17], and **Materna-Trace-3** (547 VMs, 2016) from the Materna GWA-T-13 trace. Each VM reports, at 300-second sampling, its provisioned cores, CPU capacity (MHz), memory capacity, and CPU/memory usage. These are the same traces our synthetic workload was modeled on, which makes them the natural external check; they are also among the few public traces exposing both provisioned capacity *and* usage per VM at this granularity (the Azure and Google traces bucket or omit provisioned memory [3], [8], and the Alibaba traces report container-level, normalized usage [50], [51]).

Cost model. We price VMs against a 28-type AWS `us-east-1` Linux catalog (t3/m5/c5/r5 families) whose on-demand and one-year standard no-upfront reserved rates were extracted from AWS’s Price List bulk file (snapshot 2026-06-30) and cross-checked against two independent sources; the implied reserved ratio is $\gamma = 0.63$. The *baseline* is a like-for-like lift-and-shift: each VM is assigned the cheapest type covering its *provisioned* cores and memory—the migration a team performs when it does not right-size. The *optimized* assignment solves the system’s own MILP (5) with requirements from the deployed rule (4): the 95th percentile of observed CPU (and memory) usage with $1.3\times$ headroom. No VM exceeded the largest catalog type (0 clamped). Without a budget cap the integer program decomposes into independent per-VM cheapest-covering choices, so this study exercises the system’s requirement definitions and implementation rather than the coupled, budget-constrained program.

B. Right-sizing results

Table III summarizes. On `fastStorage` the MILP cuts the baseline bill from \$128,588/month to \$85,414/month (**33.6%**), down-sizing 765 VMs while *up*-sizing 181 genuinely under-provisioned ones (the rule is not a pure cost cutter; it also fixes performance risks). The independent `Rnd` dataset gives a nearly identical relative saving (33.1%)—evidence the result is a property of the method rather than of one dataset—and `Materna`, a far more over-provisioned infrastructure, yields 74.2%. Because trace VMs are always-on, the reservation lever applies on top: pricing the optimized fleet at one-year reserved rates yields total reductions of 58.3%, 58.0%, and 83.8% respectively. The magnitudes are consistent with published production experience: at Google, autopiloted jobs run at 23% slack versus 46% for manually-managed ones [15], and a recent SAP production trace finds the large majority of VMs using well under their provisioning [5].

C. Scale, and a boundary condition: a 237k-VM Azure fleet

To probe scale and era, we repeat the right-sizing analysis on the Azure Public Dataset V2 (2019 production trace from the Resource Central repository [3]): of 2,695,548 VMs, 237,272 survive the filters (lifetime ≥ 7 days—most Azure VMs are short-lived—plus numeric capacity buckets and a valid p_{95} reading; the 10,923 VMs in the unpublishable “>24-core/>64-GB” buckets are the trace’s largest), and for each Azure publishes provisioned core/memory *buckets* and the 95th percentile of five-minute *maximum* CPU readings. Two

TABLE III
STUDY II: RIGHT-SIZING REAL VMs WITH THE SYSTEM’S MILP AGAINST A LIKE-FOR-LIKE LIFT-AND-SHIFT BASELINE, PRICED AT AWS ON-DEMAND RATES (SNAPSHOT 2026-06-30); THE LAST COLUMN ADDITIONALLY APPLIES ONE-YEAR RESERVED-INSTANCE (RI) PRICING ($\gamma = 0.63$) TO THE OPTIMIZED FLEET.

Dataset	VMs	Baseline (\$/mo)	Optimized (\$/mo)	Saving (%)	+RI (%)
Bitbrains <code>fastStorage</code>	1250	128,588	85,414	33.6	58.3
Bitbrains <code>Rnd</code>	500	44,294	29,614	33.1	58.0
Materna-Trace-3	547	44,184	11,398	74.2	83.8

semantics therefore differ from the Bitbrains/Materna study: utilization percentiles are max-based rather than mean-based, and—since no memory-usage series is published—memory is held at its provisioned bucket (never downsized). The result is an instructive *negative*: right-sizing alone is cost-negative (-3.8%; 21,772 VMs down-sized but 26,227 up-sized, since any VM with max-based p_{95} above 77% exceeds its provisioned cores under the $1.3\times$ headroom), even though the fleet carries 53% aggregate CPU slack that the memory floor and hot maxima make unreachable. The reservation lever, by contrast, still yields 34.8% on the same fleet—net of the right-sizing loss (reserved pricing alone saves $1-\gamma$ on an always-on fleet). The reading is a boundary condition for the deployed rule, invisible on traces that publish usage means: the fractile heuristic’s value is contingent on the *semantics* of the utilization metric and on which resources are measurable, whereas the γ -driven pricing lever is robust across all four datasets.

D. Forecasting real aggregate demand is hard

We repeat the walk-forward protocol of Section V-B on the `Rnd` cluster’s aggregate CPU demand (2142 hourly points over three months, mean 402.4 GHz; Fig. 7), at its native hourly scale (season 24, horizons 6–48 h) and, aligned with Study I, on the daily aggregate (horizons 7/14 d). Table IV reports the results, and they are sobering: the best hourly model reaches only 60.8% MAPE at 24 h (Prophet), the best daily model 52.1% at 7 d (ETS), and none of the tested pairs is statistically separable (ETS vs. seasonal-naive DM $p = 0.232$; Prophet vs. seasonal-naive $p = 0.235$; Prophet vs. naive $p = 0.604$; all at 24 h). The MASE values tell the structural story: all models sit near 1.0 (0.91–1.35 at 24 h), i.e. none meaningfully beats a naive benchmark, and seasonal-naive is *worse* than plain naive (MASE 1.35 vs. 0.91)—this aggregate is dominated by irregular regime shifts rather than stable daily seasonality, and 149 of 2142 hours sit above the $\mu + 2\sigma$ surge threshold. ETS interval coverage collapses to 54% (nominal 95%). We report this as a finding, not a failure: it is precisely the regime in which point-forecast-driven provisioning is fragile, and it motivates the design in which capacity is sized from the *empirical demand distribution* (quantiles, Section VII) while point forecasts serve the softer roles of trend reporting and anomaly context.

VII. STUDY III: THE PRICE OF CONSERVATISM

Sections V–VI evaluated the system as built. This section asks the converse question: *what would the theory of Section III*

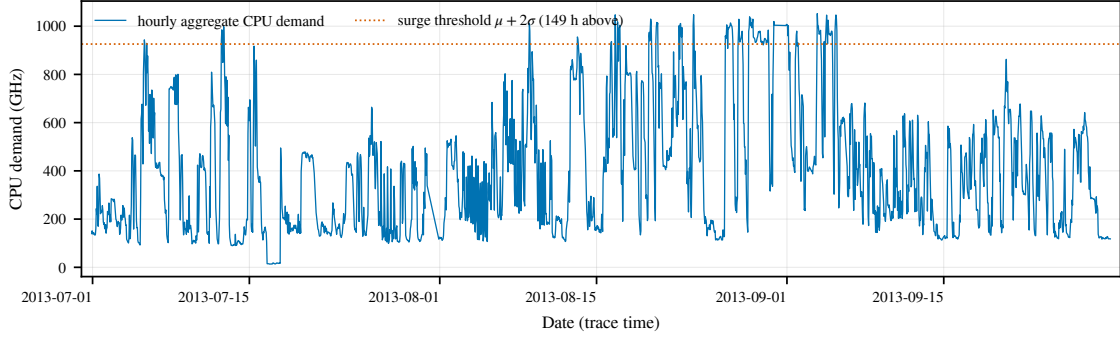


Fig. 7. Study II: hourly aggregate CPU demand of the Bitbrains Rnd cluster (500 VMs, three months). 149 hours exceed the $\mu + 2\sigma$ surge threshold; the series exhibits irregular regime shifts rather than stable seasonality.

TABLE IV

STUDY II: WALK-FORWARD MAPE (%), MEAN $\pm$ SD ACROSS FOLDS) ON THE REAL RND AGGREGATE, AT HOURLY (LEFT) AND DAILY (RIGHT) GRANULARITY. BEST MEAN PER COLUMN IN BOLD. NO TESTED PAIRWISE DIFFERENCE IS STATISTICALLY SIGNIFICANT (SECTION VI-D).

Model	$h=6\text{ h}$	$h=12\text{ h}$	$h=24\text{ h}$	$h=48\text{ h}$	$h=7\text{ d}$	$h=14\text{ d}$
ETS	48.2 $\pm$ 46.2	71.9 $\pm$ 55.4	77.0 $\pm$ 57.2	91.2 $\pm$ 47.8	52.1 $\pm$ 29.9	64.4 $\pm$ 40.9
SeasonalNaive	65.9 $\pm$ 33.6	73.7 $\pm$ 46.3	65.0 $\pm$ 40.3	65.8 $\pm$ 33.8	54.2 $\pm$ 22.6	59.7 $\pm$ 32.9
Prophet	49.0 $\pm$ 26.2	60.5 $\pm$ 30.0	60.8 $\pm$ 21.6	88.1 $\pm$ 42.3	59.8 $\pm$ 36.9	66.7 $\pm$ 30.0
Naive	57.4 $\pm$ 71.2	87.9 $\pm$ 86.1	82.3 $\pm$ 66.6	94.2 $\pm$ 55.7	53.4 $\pm$ 14.2	66.5 $\pm$ 44.5

be worth if the sizing fractile followed it? We answer with an out-of-sample policy simulation on both workloads.

A. Protocol

Demand is the daily cost series of Study I (219 training days, 146 test days) and the hourly Rnd aggregate of Study II (1285 training hours, 857 test hours), split chronologically 60/40; every policy sets its capacity from the training split only and is charged on the test split. One exception is flagged explicitly: DUAL assumes the surge *state* is observable during execution (the information structure of the state-modulated model [9]), so we also report DUALC, a causal variant with no contemporaneous knowledge. Costs follow the average-cost model (1) in units of the on-demand rate: static reserved capacity K costs $\gamma K + \mathbb{E}(D - K)^+$ per period, so no price catalog is needed and results are reported as a percentage of the *on-demand-only* baseline. We sweep $\gamma \in [0.50, 0.95]$ and highlight $\gamma^*=0.63$, the ratio of both our catalog and current AWS one-year pricing (Sections IV–VI). Seven policies compete:

- **OD**: on-demand only ($K=0$); the normalization baseline.
- **HEUR**: the deployed fractile rule (4) read as a reservation-sizing policy, $K = q_{0.95}(D_{\text{train}}) \times 1.3$. On these workloads this covers the 99th (synthetic) and 100th (trace) percentile of training demand.
- **NV**: the newsvendor optimum of Remark 1, the empirical $(1-\gamma)$ training quantile—the data-driven newsvendor estimator [18], [19].
- **SAA**: a two-stage stochastic program (first stage: K ; second stage: per-scenario spillover) solved by sample-average approximation over the training periods with the

TABLE V

COST-OPTIMAL RESERVED SERVICE LEVEL $1 - \gamma$ VS. THE DEPLOYED $q=0.95$ HEURISTIC, BY RESERVED/ON-DEMAND RATIO $\gamma = w_T/w_0$.

$\gamma = w_T/w_0$	0.60	0.70	0.80	0.90
optimal fractile $1 - \gamma$	0.40	0.30	0.20	0.10
deployed fractile q	0.95	0.95	0.95	0.95

same PuLP/CBC toolchain as the system’s MILP; this is the classical SP formulation of cloud reservation [10], [20] serving as an independent baseline.

- **DUAL**: the dual-capacity (K_0, K_1) SAA of (1), with surge periods labeled by the training $\mu + 2\sigma$ threshold; base capacity K_0 is always reserved, supplementary $K_1 - K_0$ only during surges. This is a static step toward the (s, S) supplementary-contract dynamics of [9]; because it assumes the surge state is observable at execution time, it is an upper bound on implementable value.
- **DUALC**: the same (K_0, K_1) contracts activated *causally*—the supplementary tier is on in period t iff period $t-1$ exceeded the surge threshold. No contemporaneous knowledge.
- **LB**: a clairvoyant lower bound, $\gamma \mathbb{E}[D_{\text{test}}]$ (reserve exactly the realized demand each period; ignores commitment inflexibility).

Table V restates the analytical gap the simulation quantifies: across realistic discounts the deployed fractile (0.95) sits far above the cost-optimal $1 - \gamma$.

B. Results

Fig. 8 (synthetic) and Fig. 9 (real trace) plot realized test cost against γ ; Tables VI–VII tabulate four discount levels. Four findings:

F1 — the conservative fractile forfeits the discount. At $\gamma^*=0.63$, HEUR realizes 101.8% of the on-demand baseline on the synthetic account and 153.5% on the real trace—at and beyond break-even. Reserving to the 99th–100th percentile of a skewed demand distribution commits far more capacity than the mean demand uses: on the synthetic account HEUR crosses break-even at $\gamma \approx 0.62$, and on the trace it exceeds the on-demand bill across the entire sweep. In dollars, on

the synthetic account the rule would pay \$2,432.01/month against \$2,388.62 on-demand, while the newsvendor fractile pays \$1,757.46—a \$674.55/month *price of conservatism* (28.2 percentage points; 67.4 points on the trace). We emphasize the framing: the deployed *system* does not size reservations this way—it right-sizes instance types with (4) and gates the reservation switch on always-on usage (Section IV)—and F1 is precisely the quantitative reason that design choice is correct. F1 is thus a cautionary out-of-sample bound on naively transferring the right-sizing fractile to commitment sizing—a transfer that is tempting precisely because the percentile idiom is standard for capacity limits [3], [15], while commercial commitment recommenders instead maximize savings over replayed historical usage [45]–[47]; F1 quantifies why that is the right instinct.

F2 — the newsvendor fractile is robust out of sample. NV realizes 73.6% (synthetic) and 86.2% (trace) of the on-demand baseline at γ^* —a 26.4%/13.8% saving from a one-line, distribution-free rule—even though the trace’s test window is substantially surlier than its training window (124 vs. 54 surge hours under the training-split threshold; the full-series threshold of Section VI flags 149). The ordering is insensitive to the split choice: across train fractions 0.5–0.7 the newsvendor saving stays within 26.1–26.4% (synthetic) and 12.5–16.0% (trace).

F3 — the two-tier structure pays exactly when surge states persist. DUAL improves on NV by only 0.8 points on the tame synthetic series (72.8% vs. 73.6%) but by 8.5 points on the real trace (77.7% vs. 86.2%): the base commitment stays at the newsvendor level ($K_0=68.5$ vs. single $K=68.8$ \$/day on the synthetic account) while a supplementary tier ($K_1=195.7$) is paid for only during surge periods—capacity a single static K must either commit to permanently or forgo. DUAL’s observable-state assumption makes it an upper bound; the causal variant tells the sharper story. On the real trace, where surge episodes persist across hours, activating on the previous hour’s signal retains nearly all of the value (DUALC +8.1 of the +8.5 points); on the synthetic account, whose surges are isolated single days, DUALC arrives a day late and realizes 74.6%—*worse* than NV. Surge persistence, not burstiness alone, is the enabling condition. On the trace the finding is robust to the labeling threshold (sweeping $\mu+k\sigma$, $k \in \{1.5, 2, 2.5\}$, the causal gain stays positive at 2.7–8.1 points) and degrades gracefully under contract-term friction: with contracts sized ignoring the term constraint, a minimum supplementary term of 6 or 24 hours reduces the causal gain to 6.6 and 4.1 points, while a week-long minimum term destroys it (−7.9)—quantifying, on real demand, why the cancellation flexibility at the heart of [9] matters. This is, to our knowledge, the first out-of-sample quantification of the value of the base/supplementary contract structure of [9] on real cloud telemetry, and it supports implementing the full (s, S) dynamics as the highest-value system extension.

F4 — the SP baseline recovers the closed form. The SAA program’s capacity matches the newsvendor quantile to within 0.57% (synthetic) and 0.12% (trace) across the sweep—as theory requires, since SAA of the same empirical distribution optimizes the same objective—which cross-validates the closed-form policy, our simulator, and the SP implementation, and

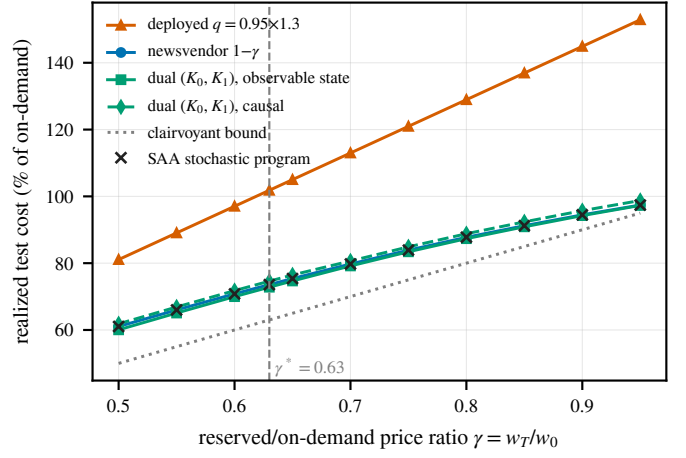


Fig. 8. Study III (synthetic account): realized out-of-sample cost, as % of on-demand-only, vs. reserved discount γ . The deployed $q=0.95 \times 1.3$ fractile read as reservation sizing crosses break-even at $\gamma \approx 0.62$; the newsvendor fractile and its SAA twin track the clairvoyant bound.

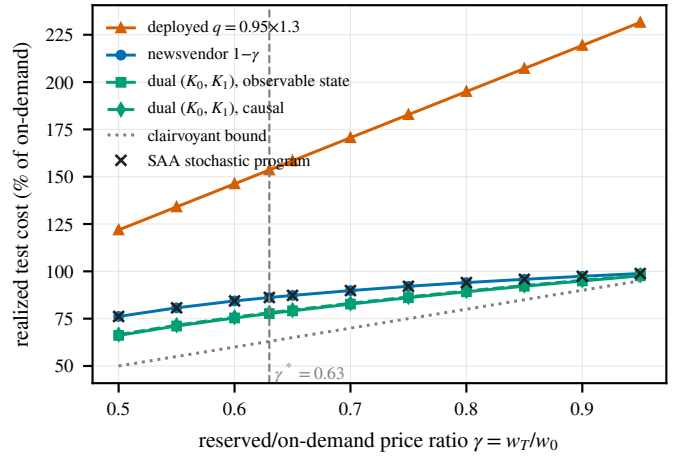


Fig. 9. Study III (real Bitbrains Rnd demand): realized out-of-sample cost, as % of on-demand-only, vs. reserved discount γ (cf. Fig. 8). On real demand with persistent surges both dual-capacity variants open a clear gap over single-capacity newsvendor sizing.

confirms that nothing here hinges on solver choice. Because savings maximization over replayed historical usage is also the computation commercial recommenders document [45]–[47], F4 doubles as an indirect validation of that practice: replay-and-maximize recovers the newsvendor optimum, and both differ sharply from fractile-based sizing (F1).

C. Implications for the deployed system

The simulation prescribes three concrete changes, in decreasing order of value. First, any lever that sizes *commitments* should use the γ -aware newsvendor fractile $1 - \gamma$, not a fixed high percentile; the required inputs (the reserved ratio and the empirical demand distribution) are already in the system’s database. Second, for workloads with persistent surges the base/supplementary split is worth implementing—on real demand the observable-state policy captured 37% of the remaining gap between NV and the clairvoyant bound,

TABLE VI

STUDY III (SYNTHETIC): REALIZED TEST COST AS % OF ON-DEMAND-ONLY, BY POLICY AND RESERVED DISCOUNT γ . BOLD: BEST AMONG POLICIES REQUIRING NO CONTEMPORANEOUS SURGE-STATE KNOWLEDGE (THE OBSERVABLE-STATE DUAL AND THE CLAIRVOYANT BOUND ARE EXCLUDED FROM BOLDING).

Policy	$\gamma=0.60$	$\gamma=0.70$	$\gamma=0.80$	$\gamma=0.90$
deployed $q=0.95 \times 1.3$	97.0	113.0	128.9	144.9
newsvendor $1-\gamma$	70.8	79.7	87.7	94.5
SAA stochastic program	70.8	79.8	87.8	94.5
dual (K_0, K_1) , obs. state	70.0	79.2	87.3	94.2
dual (K_0, K_1) , causal	71.8	80.6	88.8	95.7
clairvoyant bound	60.0	70.0	80.0	90.0

TABLE VII

STUDY III (REAL BITBRAINS RND DEMAND): REALIZED TEST COST AS % OF ON-DEMAND-ONLY, BY POLICY AND RESERVED DISCOUNT γ (CF. TABLE VI).

Policy	$\gamma=0.60$	$\gamma=0.70$	$\gamma=0.80$	$\gamma=0.90$
deployed $q=0.95 \times 1.3$	146.2	170.6	195.0	219.3
newsvendor $1-\gamma$	84.4	89.8	94.1	97.4
SAA stochastic program	84.4	89.8	94.1	97.4
dual (K_0, K_1) , obs. state	75.4	82.7	89.2	94.9
dual (K_0, K_1) , causal	75.8	83.1	89.6	95.3
clairvoyant bound	60.0	70.0	80.0	90.0

and the causal activation rule retained 35%—which is the static core of the reference’s (s, S) policy; the dynamic cancellation/renewal thresholds remain future work. Third, the fixed $q=0.95 \times 1.3$ rule remains appropriate where it is actually deployed (sizing instance types against performance risk); the system’s separation of concerns—percentile for capacity, always-on gating for commitment—is validated by F1.

VIII. THREATS TO VALIDITY

Workload provenance. Study I uses a synthetic, single-account dataset that ships with the system; its dollar figures are method-illustrative, not measured savings on production infrastructure. Study II mitigates this with real telemetry, but from two providers with traces collected in 2013 (Bitbrains) and 2016 (Materna), each spanning one to three months; modern cloud usage is burstier and more heterogeneous [8], which plausibly *strengthens* the case for surge-aware policies (F3) but is untested here; the Azure arm (2019) is newer but bucketed. Validation on a post-2019 per-VM trace (e.g., [5]) and on a live account remain open.

Trace cost model. Study II prices 2013–2019 VMs at 2026 AWS rates under a like-for-like mapping (cheapest type covering provisioned capacity), ignores licensing, storage, network egress, and sustained-use or negotiated discounts, and assumes VMs are always-on when applying reserved rates (true for the trace populations studied). The baseline is a *naïve* lift-and-shift; a team that already right-sizes would see smaller gains. Savings percentages are therefore comparable across datasets and methods, but absolute dollars are indicative only. The Azure arm (Section VI-C) is additionally *not* like-for-like with the others: capacities are bucketed, utilization percentiles

are max-based, memory is never downsized, and the seven-day lifetime filter (which drops 91% of rows, with lifetimes right-censored at the 30-day trace end) restricts it to the long-running population that right-sizing targets; its 34.8% reserved figure is an upper bound, since lifetimes are right-censored at the 30-day trace end while a one-year commitment must stay utilized.

Policy-simulation scope. Study III sizes a *single aggregate* resource, abstracting instance granularity, contract term lengths, partial upfronts, and cancellation; its LB ignores commitment inflexibility entirely. On the synthetic arm the aggregate is the whole daily bill, which includes non-reservable services (S3, Lambda, NAT), overstating the reservable base; the trace arm (pure CPU demand) does not share this caveat and shows the same qualitative ordering. HEUR applies the deployed fractile to a decision the deployed system does not make with it (Section VII states this framing explicitly); DUAL assumes an observable surge state (DUALC removes the assumption), and both depend on the surge labeling—F3 sweeps the threshold multiplier (trace gains remain positive) and the minimum contract term (gains vanish at week-long terms). All conclusions rest on chronological splits of one series per workload; the train-fraction sweep in F2 probes, but does not eliminate, this dependence. The (s, S) cancellation/renewal dynamics of [9] are not simulated—DUAL/DUALC are their static core only.

Statistical power. Walk-forward evaluation yields 6–9 folds per configuration (as few as 6 at the longest synthetic horizon); at that size only large accuracy differences are detectable, and the ETS-vs-seasonal-naïve comparison is inconclusive in both studies (Section V-B, Section VI-D). MAPE excludes zero-cost periods and is sensitive to surge periods; MASE is reported alongside for scale-free comparison. Prediction-interval semantics differ across models (ETS uses $\pm 1.96\sigma$ of residuals), and Prophet’s intervals undercover on both workloads. SARIMAX is omitted from the sweeps for runtime.

System gap. The deployed system realizes *static* fixed-fractile sizing ($q_{0.95} \times 1.3$) with always-on reservation gating—not the γ -aware newsvendor sizing that Section VII recommends, and not the full (s, S) dynamics; the reserved switch is a per-resource, all-or-nothing one-year commitment. Several catalog entries lack vCPU/memory specifications and are excluded as MILP candidates, slightly shrinking the feasible set. The LLM advisor is not evaluated here.

Baselines. The stochastic-programming (SP) baseline (SAA) is internal, and—as F4 shows—coincides with the newsvendor quantile by construction; it implements the replay-and-maximize computation that commercial recommenders document [45], [47], but not their production implementations, so it validates correctness rather than providing an adversarial competitor. We do not compare head-to-head against the tools themselves (e.g., AWS Compute Optimizer or Cost Explorer’s recommenders) on the same workload; our claims are integration, validation, and gap quantification, not superiority over existing tooling.

IX. CONCLUSION

This paper turned the capacity-reservation policy of [9] into a deployable, data-driven system, and then measured

the bridge from both ends. On a synthetic AWS account the pipeline surfaces \$588.40–\$590.40/month (27.0% of the bill), concentrated in MILP right-sizing and the reserved switch. On real telemetry from 2,297 VMs across two providers, the same MILP cuts a lift-and-shift baseline by 33.1–74.2% (58.0–83.8% with one-year reservations); a 237,272-VM Azure fleet exposes the rule’s boundary condition—max-based percentiles and unmeasurable memory flip right-sizing cost-negative while reservations still save 34.8%—and honest forecasting results show real aggregate demand is dominated by regime shifts that defeat point forecasts, supporting quantile-based sizing. An out-of-sample policy simulation then quantified the theory’s value: at the measured reserved discount, sizing commitments at the industry’s conservative 95th-percentile idiom would forfeit the entire discount (101.8%–153.5% of on-demand cost), the newsvendor fractile captures 73.6%–86.2%, a stochastic-programming baseline independently recovers the same quantile, and the reference’s base/supplementary contract structure—worth little on tame demand—adds 8.5 points on real bursty demand, 8.1 of which survive under causal activation. Implementing the full (s, S) cancellation/renewal dynamics, testing time-series foundation models, and validating on a live account and post-2019 per-VM traces [5] are the natural next steps.

ACKNOWLEDGMENTS

The Bitbrains GWA-T-12 and Materna GWA-T-13 traces are used courtesy of Bitbrains IT Services Inc., Materna GmbH, and the Grid Workloads Archive [17]; we thank the @Large research group for mirroring the archive.

REFERENCES

- [1] Flexera, “State of the cloud report,” Technical report, Flexera Inc., <https://www.flexera.com/>, 2024, accessed 2026.
- [2] B. Everman, M. Gao, and Z. Zong, “Evaluating and reducing cloud waste and cost—A data-driven case study from Azure workloads,” *Sustainable Computing: Informatics and Systems*, vol. 35, p. 100708, 2022.
- [3] E. Cortez, A. Bonde, A. Muzio, M. Russinovich, M. Fontoura, and R. Bianchini, “Resource Central: Understanding and predicting workloads for improved resource management in large cloud platforms,” in *Proceedings of the 26th Symposium on Operating Systems Principles (SOSP)*, 2017, pp. 153–167.
- [4] B. Reidys, P. Zardoshti, Í. Goiri, C. Irvine, D. S. Berger, H. Ma, K. Arya, E. Cortez, T. Stark, E. Bak, M. Iyigun, S. Novakovic, L. Hsu, K. Trueba, A. Pan, C. Bansal, S. Rajmohan, J. Huang, and R. Bianchini, “Coach: Exploiting temporal patterns for all-resource oversubscription in cloud platforms,” in *Proc. 30th ACM Int. Conf. on Architectural Support for Programming Languages and Operating Systems (ASPLOS), Volume 1*. New York, NY, USA: ACM, 2025, pp. 164–181.
- [5] A. Uhlig, I. Braun, and M. Wählich, “The SAP cloud infrastructure dataset: A reality check of scheduling and placement of VMs in cloud computing,” in *Proc. 2025 ACM Internet Measurement Conference (IMC)*. ACM, 2025, pp. 746–760.
- [6] M. Armbrust, A. Fox, R. Griffith, A. D. Joseph, R. Katz, A. Konwinski, G. Lee, D. Patterson, A. Rabkin, I. Stoica, and M. Zaharia, “Above the clouds: A Berkeley view of cloud computing,” EECS Department, University of California, Berkeley, Tech. Rep. UCB/EECS-2009-28, 2009.
- [7] S. Shen, V. van Beek, and A. Iosup, “Statistical characterization of business-critical workloads hosted in cloud datacenters,” in *2015 15th IEEE/ACM Int. Symp. on Cluster, Cloud and Grid Computing (CCGrid)*, 2015, pp. 465–474.
- [8] M. Tirmazi, A. Barker, N. Deng, M. E. Haque, Z. G. Qin, S. Hand, M. Harchol-Balter, and J. Wilkes, “Borg: The next generation,” in *Proceedings of the Fifteenth European Conference on Computer Systems (EuroSys)*, 2020, pp. 1–14.
- [9] S. Chen, J. Lei, and K. Moinsadeh, “Cost optimization in cloud computing: Capacity reservation for intermittent random demand surges,” *Production and Operations Management*, vol. 33, no. 6, pp. 1265–1284, 2024, also available as SSRN working paper 3784812.
- [10] S. Chaisiri, B.-S. Lee, and D. Niyato, “Optimization of resource provisioning cost in cloud computing,” *IEEE Transactions on Services Computing*, vol. 5, no. 2, pp. 164–177, 2012.
- [11] W. Wang, B. Liang, and B. Li, “Optimal online multi-instance acquisition in IaaS clouds,” *IEEE Transactions on Parallel and Distributed Systems*, vol. 26, no. 12, pp. 3407–3419, 2015.
- [12] L. Hosseini, S. Tang, V. Mookerjee, and C. Sriskandarajah, “A switch in time saves the dime: A model to reduce rental cost in cloud computing,” *Information Systems Research*, vol. 31, no. 3, pp. 753–775, 2020.
- [13] D. a. F. Kirchoff, V. Meyer, R. N. Calheiros, and C. A. F. De Rose, “Evaluating machine learning prediction techniques and their impact on proactive resource provisioning for cloud environments,” *The Journal of Supercomputing*, vol. 80, no. 15, pp. 21 920–21 951, 2024.
- [14] F. Agulló, A. Gutierrez-Torre, J. Torres, and J. L. Berral, “Enhancing the output of time series forecasting algorithms for cloud resource provisioning,” *Future Generation Computer Systems*, vol. 170, p. 107833, 2025.
- [15] K. Rządca, P. Findeisen, J. Swiderski, P. Zych, P. Broniek, J. Kusmierek, P. Nowak, B. Strack, P. Witusowski, S. Hand, and J. Wilkes, “Autopilot: Workload autoscaling at Google,” in *Proceedings of the Fifteenth European Conference on Computer Systems (EuroSys ’20)*. ACM, 2020, pp. 1–16.
- [16] J. Cahoon, W. Wang, Y. Zhu, K. Lin, S. Liu, R. Truong, N. Singh, C. Wan, A. Ciortea, S. Narasimhan, and S. Krishnan, “Doppler: Automated SKU recommendation in migrating SQL workloads to the cloud,” *Proceedings of the VLDB Endowment*, vol. 15, no. 12, pp. 3509–3521, 2022.
- [17] A. Iosup, H. Li, M. Jan, S. Anoop, C. Dumitrescu, L. Wolters, and D. H. J. Epema, “The Grid workloads archive,” *Future Generation Computer Systems*, vol. 24, no. 7, pp. 672–686, 2008.
- [18] K. J. Arrow, T. Harris, and J. Marschak, “Optimal inventory policy,” *Econometrica*, vol. 19, no. 3, pp. 250–272, 1951.
- [19] G.-Y. Ban and C. Rudin, “The big data newsvendor: Practical insights from machine learning,” *Operations Research*, vol. 67, no. 1, pp. 90–108, 2019.
- [20] K. Bülbül, N. Noyan, and H. Erol, “Multi-stage stochastic programming models for provisioning cloud computing resources,” *European Journal of Operational Research*, vol. 288, no. 3, pp. 886–901, 2021.
- [21] R. J. Hyndman and A. B. Koehler, “Another look at measures of forecast accuracy,” *International Journal of Forecasting*, vol. 22, no. 4, pp. 679–688, 2006.
- [22] F. X. Diebold and R. S. Mariano, “Comparing predictive accuracy,” *Journal of Business & Economic Statistics*, vol. 13, no. 3, pp. 253–263, 1995.
- [23] D. Harvey, S. Leybourne, and P. Newbold, “Testing the equality of prediction mean squared errors,” *International Journal of Forecasting*, vol. 13, no. 2, pp. 281–291, 1997.
- [24] G. D. Eppen and A. V. Iyer, “Backup agreements in fashion buying—the value of upstream flexibility,” *Management Science*, vol. 43, no. 11, pp. 1469–1484, 1997.
- [25] M. Erkoç and S. D. Wu, “Managing high-tech capacity expansion via reservation contracts,” *Production and Operations Management*, vol. 14, no. 2, pp. 232–251, 2005.
- [26] L. Huang, J.-S. Song, and J. Tong, “Supply chain planning for random demand surges: Reactive capacity and safety stock,” *Manufacturing & Service Operations Management*, vol. 18, no. 4, pp. 509–524, 2016.
- [27] J. Dong and R. Ibrahim, “Managing supply in the on-demand economy: Flexible workers, full-time employees, or both?” *Operations Research*, vol. 68, no. 4, pp. 1238–1264, 2020.
- [28] J.-S. Song and P. Zipkin, “Inventory control in a fluctuating demand environment,” *Operations Research*, vol. 41, no. 2, pp. 351–370, 1993.
- [29] W. Fischer and K. Meier-Hellstern, “The Markov-modulated Poisson process (MMPP) cookbook,” *Performance Evaluation*, vol. 18, no. 2, pp. 149–171, 1993.
- [30] G. Huang, G. Yu, and X. Zhao, “Refining data-driven upfront reservation discount pricing via inverse inferring newsvendor transactions,” *Production and Operations Management*, vol. 35, no. 5, pp. 1766–1784, 2025.
- [31] E. Furman and A. Diamant, “Optimal capacity planning for cloud service providers with periodic, time-varying demand,” *European Journal of Operational Research*, vol. 322, no. 1, pp. 133–146, 2025.
- [32] S. Mandal, G. Maji, S. Khatua, and R. K. Das, “Cost minimizing reservation and scheduling algorithms for public clouds,” *IEEE Transactions on Cloud Computing*, vol. 11, no. 2, pp. 1365–1380, 2023.

- [33] Y. Shin, C. Lee, G. Lee, and H.-C. An, “Improved learning-augmented algorithms for the multi-option ski rental problem via best-possible competitive analysis,” in *Proceedings of the 40th International Conference on Machine Learning (ICML)*, ser. Proceedings of Machine Learning Research, vol. 202. PMLR, 2023, pp. 31 539–31 561.
- [34] K. Zhang, Z. Liu, N. Choi, and B. Ji, “Learning-augmented online algorithm for two-level ski-rental problem,” in *Proceedings of the 38th AAAI Conference on Artificial Intelligence*, vol. 38, no. 18, 2024, pp. 20 293–20 300.
- [35] P. Ambati, D. Irwin, and P. Shenoy, “No reservations: A first look at Amazon’s reserved instance marketplace,” in *12th USENIX Workshop on Hot Topics in Cloud Computing (HotCloud 20)*, 2020.
- [36] M. C. Cohen, P. W. Keller, V. Mirrokni, and M. Zadimoghaddam, “Overcommitment in cloud services: Bin packing with chance constraints,” *Management Science*, vol. 65, no. 7, pp. 3255–3271, 2019.
- [37] M. A. Núñez, X. Bai, and L. Du, “Leveraging slack capacity in IaaS contract cloud services,” *Production and Operations Management*, vol. 30, no. 4, pp. 883–901, 2021.
- [38] M. E. Arbabian, S. Chen, and K. Moinsadeh, “Capacity expansions with bundled supplies of attributes: An application to server procurement in cloud computing,” *Manufacturing & Service Operations Management*, vol. 23, no. 1, pp. 191–209, 2021.
- [39] S. Chen, K. Moinsadeh, and Y. Tan, “Discount schemes for the preemptible service of a cloud platform with unutilized capacity,” *Information Systems Research*, vol. 32, no. 3, pp. 967–986, 2021.
- [40] S. Chen, H. Lee, and K. Moinsadeh, “Pricing schemes in cloud computing: Utilization-based versus reservation-based,” *Production and Operations Management*, vol. 28, no. 1, pp. 82–102, 2019.
- [41] S. Deochake, “ABACUS: A FinOps service for cloud cost optimization,” arXiv preprint arXiv:2501.14753, 2024.
- [42] M. Alves and F. Morais, “IaaS capacity planning with multiple pricing models using predictive heuristics,” in *2025 IEEE International Conference on Cloud Computing Technology and Science (CloudCom)*. Shenzhen, China: IEEE, 2025, pp. 1–8.
- [43] S. Musa and D. G. Down, “Cost-effective cloud resource provisioning using linear regression,” in *2024 IEEE International Conference on Cloud Computing Technology and Science (CloudCom)*. Abu Dhabi, United Arab Emirates: IEEE, 2024, pp. 176–183.
- [44] M. A. Anjum, “Bridging FinOps practice and machine learning research: a systematic review of cloud cost optimization,” *PeerJ Computer Science*, vol. 12, p. e3964, 2026.
- [45] Amazon Web Services, “Accessing reservation recommendations,” <https://docs.aws.amazon.com/cost-management/latest/userguide/rri-recommendations.html>, 2026, accessed 2026-07-03.
- [46] —, “Understanding your recommendation calculations,” <https://docs.aws.amazon.com/savingsplans/latest/userguide/sp-rec-calculations.html>, 2026, accessed 2026-07-03.
- [47] Microsoft, “Azure reservation recommendations,” <https://learn.microsoft.com/en-us/azure/cost-management-billing/reservations/reserved-instance-purchase-recommendations>, 2026, accessed 2026-07-03.
- [48] N. J. Yadwadkar, B. Hariharan, J. E. Gonzalez, B. Smith, and R. H. Katz, “Selecting the best VM across multiple public clouds: A data-driven performance modeling approach,” in *Proceedings of the 2017 Symposium on Cloud Computing (SoCC ’17)*. New York, NY, USA: ACM, 2017, pp. 452–465.
- [49] N. Aydın, İ. Muter, and Ş. İ. Birbil, “Multi-objective temporal bin packing problem: An application in cloud computing,” *Computers & Operations Research*, vol. 121, p. 104959, 2020.
- [50] C. Lu, K. Ye, G. Xu, C.-Z. Xu, and T. Bai, “Imbalance in the cloud: An analysis on Alibaba cluster trace,” in *2017 IEEE International Conference on Big Data (Big Data)*, 2017, pp. 2884–2892.
- [51] J. Guo, Z. Chang, S. Wang, H. Ding, Y. Feng, L. Mao, and Y. Bao, “Who limits the resource efficiency of my datacenter: An analysis of Alibaba datacenter traces,” in *Proceedings of the IEEE/ACM International Symposium on Quality of Service (IWQoS)*, 2019, pp. 1–10.
- [52] I. Pintye, J. Kovács, and R. Lovas, “Enhancing machine learning-based autoscaling for cloud resource orchestration,” *Journal of Grid Computing*, vol. 22, no. 4, p. 68, 2024.
- [53] H. Ahmad, C. Treude, M. Wagner, and C. Szabo, “Towards resource-efficient reactive and proactive auto-scaling for microservice architectures,” *Journal of Systems and Software*, vol. 225, p. 112390, 2025.
- [54] D. Zou, W. Lu, Z. Zhu, X. Lu, J. Zhou, X. Wang, K. Liu, K. Wang, R. Sun, and H. Wang, “Optscaler: A collaborative framework for robust autoscaling in the cloud,” *Proceedings of the VLDB Endowment*, vol. 17, no. 12, pp. 4090–4103, 2024.
- [55] Z. Wang, S. Zhu, J. Li, W. Jiang, K. K. Ramakrishnan, Y. Zheng, M. Yan, X. Zhang, and A. X. Liu, “Deepscaling: Microservices autoscaling for stable CPU utilization in large scale cloud systems,” in *Proceedings of the 13th Symposium on Cloud Computing (SoCC ’22)*. ACM, 2022, pp. 16–30.
- [56] V. Sachidananda and A. Sivaraman, “Erlang: Application-aware autoscaling for cloud microservices,” in *Proceedings of the Nineteenth European Conference on Computer Systems (EuroSys ’24)*. ACM, 2024, pp. 888–923.
- [57] P. R. Winters, “Forecasting sales by exponentially weighted moving averages,” *Management Science*, vol. 6, no. 3, pp. 324–342, 1960.
- [58] R. J. Hyndman and G. Athanasopoulos, *Forecasting: Principles and Practice*, 3rd ed. OTexts, Melbourne, Australia, 2021, <https://otexts.com/fpp3/>.
- [59] R. J. Hyndman and Y. Khandakar, “Automatic time series forecasting: The forecast package for R,” *Journal of Statistical Software*, vol. 27, no. 3, pp. 1–22, 2008.
- [60] S. J. Taylor and B. Letham, “Forecasting at scale,” *The American Statistician*, vol. 72, no. 1, pp. 37–45, 2018.
- [61] S. Seabold and J. Perktold, “statsmodels: Econometric and statistical modeling with Python,” in *Proc. 9th Python in Science Conference (SciPy 2010)*, 2010, pp. 92–96.
- [62] S. Ahmad, A. Lavin, S. Purdy, and Z. Agha, “Unsupervised real-time anomaly detection for streaming data,” *Neurocomputing*, vol. 262, pp. 134–147, 2017.
- [63] M. Xu, C. Song, H. Wu, S. S. Gill, K. Ye, and C. Xu, “esDNN: Deep neural network based multivariate workload prediction in cloud computing environments,” *ACM Transactions on Internet Technology*, vol. 22, no. 3, pp. 1–24, 2022.
- [64] J. Chen, H. Ye, F. Jiang, X. He, T. Zhang, J. Chen, and X. Gao, “Fremr: Lightweight and effective frequency transformer for workload forecasting in cloud services,” arXiv preprint arXiv:2507.12908, 2025.
- [65] A. F. Ansari, L. Stella, C. Turkmen, X. Zhang, P. Mercado, H. Shen, O. Shchur, S. S. Rangapuram, S. Pineda Arango, S. Kapoor, J. Zschneider, D. C. Maddix, H. Wang, M. W. Mahoney, K. Torkkola, A. Gordon Wilson, M. Bohlke-Schneider, and Y. Wang, “Chronos: Learning the language of time series,” *Transactions on Machine Learning Research*, 2024, arXiv:2403.07815.
- [66] A. Das, W. Kong, R. Sen, and Y. Zhou, “A decoder-only foundation model for time-series forecasting,” in *Proc. 41st Int. Conf. Machine Learning (ICML)*, ser. Proceedings of Machine Learning Research, vol. 235. PMLR, 2024, arXiv:2310.10688.
- [67] G. Woo, C. Liu, A. Kumar, C. Xiong, S. Savarese, and D. Sahoo, “Unified training of universal time series forecasting transformers,” in *Proc. 41st Int. Conf. Machine Learning (ICML)*, ser. Proceedings of Machine Learning Research, vol. 235. PMLR, 2024, arXiv:2402.02592.
- [68] G. Woo, C. Liu, A. Kumar, and D. Sahoo, “Pushing the limits of pre-training for time series forecasting in the CloudOps domain,” arXiv preprint arXiv:2310.05063, 2023.
- [69] A. Zeng, M. Chen, L. Zhang, and Q. Xu, “Are transformers effective for time series forecasting?” in *Proc. AAAI Conf. Artificial Intelligence*, vol. 37, no. 9, 2023, pp. 11 121–11 128.
- [70] A. Lackinger, A. Morichetta, and S. Dustdar, “Time series predictions for cloud workloads: A comprehensive evaluation,” in *2024 IEEE International Conference on Service-Oriented System Engineering (SOSE)*, 2024, pp. 36–45.
- [71] L. J. Tashman, “Out-of-sample tests of forecasting accuracy: an analysis and review,” *International Journal of Forecasting*, vol. 16, no. 4, pp. 437–450, 2000.
- [72] C. Bergmeir and J. M. Benítez, “On the use of cross-validation for time series predictor evaluation,” *Information Sciences*, vol. 191, pp. 192–213, 2012.
- [73] S. Mitchell, M. O’Sullivan, and I. Dunning, “PuLP: A linear programming toolkit for Python,” Optimization Online technical report, The University of Auckland, 2011, <https://optimization-online.org/2011/09/3178/>.
- [74] F. X. Diebold, “Comparing predictive accuracy, twenty years later: A personal perspective on the use and abuse of Diebold–Mariano tests,” *Journal of Business & Economic Statistics*, vol. 33, no. 1, pp. 1–9, 2015.
- [75] H. Hewamalage, K. Ackermann, and C. Bergmeir, “Forecast evaluation for data scientists: common pitfalls and best practices,” *Data Mining and Knowledge Discovery*, vol. 37, no. 2, pp. 788–832, 2023.